

گزارش کار

پیاده سازی دیجیتال بازی رومیزی Unmatched

یک موتور بازی کارتی داده محور با ++C17، به همراه لایه نمایش گرافیکی مبتنی بر raylib

کیارش حبیبی
یوسف پاک

تابستان ۱۴۰۴

۱. مقدمه

این گزارش، مستندسازی کامل پروژه‌ی «Unmatched Card Game Engine» است؛ یک پیاده‌سازی نرم‌افزاری از بازی رومیزی محبوب Unmatched – با دو شخصیت Dracula و Sherlock Holmes – که با زبان C++ و کتابخانه‌ی گرافیکی سبک raylib نوشته شده است. هدف اصلی پروژه، طراحی و پیاده‌سازی یک موتور بازی ماژولار، قابل‌توسعه و داده‌محور بوده است؛ به‌گونه‌ای که قوانین، شخصیت‌ها، کارت‌ها و نقشه‌های بازی مستقیماً در کد ++C هاردکد نشوند، بلکه از فایل‌های JSON زیر پوشه‌ی data/ خوانده شوند.

این رویکرد داده‌محور، مزیت مهمی دارد: افزودن یک قهرمان، کارت یا نقشه‌ی جدید به بازی، صرفاً به معنای افزودن یک یا چند فایل JSON است و نیازی به تغییر، کامپایل مجدد یا حتی درک عمیق از کد منطق بازی ندارد. این جداسازی محتوا از کد، یکی از تصمیم‌های معماری محوری این پروژه بوده و در بخش‌های بعدی به‌تفصیل بررسی می‌شود.

در ادامه‌ی این گزارش، ابتدا اهداف پروژه را مرور می‌کنیم، سپس معماری کلی و لایه‌بندی آن، الگوهای طراحی به‌کاررفته، مکانیزم مدیریت خطا، نحوه‌ی build و اجرای پروژه، و در نهایت ساختار پوشه‌ها و توضیح فایل‌به‌فایل آن را ارائه می‌دهیم. گزارش با جمع‌بندی، محدودیت‌های شناخته‌شده و فهرست منابع مورد استفاده در پروژه به پایان می‌رسد.

۲. اهداف پروژه

- پیاده‌سازی کامل قوانین بازی Unmatched شامل حرکت جنگجویان، نبرد هم‌زمان دو کارته، اثرات کارت‌ها و تشخیص شرایط پیروزی.
- طراحی معماری‌ای که منطق بازی (لایه‌ی Model/Engine) به‌طور کامل از رابط کاربری (لایه‌ی View) جدا باشد، تا تغییر در نحوه‌ی رسم صفحه هیچ تأثیری بر قوانین بازی نداشته باشد.
- داده‌محور کردن کامل محتوای بازی؛ به‌گونه‌ای که اضافه‌کردن قهرمان یا کارت جدید فقط با افزودن فایل JSON امکان‌پذیر باشد، بدون هیچ تغییری در کد ++C.
- طراحی یک سلسله‌مراتب استثنا (Exception) اختصاصی برای مدیریت خطاهای فایل، JSON و داده به‌شکلی ایمن، قابل‌ردیابی و قابل‌بازایی در زمان اجرا.
- استفاده از الگوهای طراحی استاندارد (Factory، Observer، Singleton، Strategy، Dependency Injection) برای افزایش توسعه‌پذیری، خوانایی و نگهداری‌پذیری کد.
- فراهم‌کردن یک تجربه‌ی کاربری گرافیکی کامل با raylib، شامل منوی اصلی، صفحه‌ی انتخاب قهرمان، و صحنه‌ی اصلی بازی با پشتیبانی از صدا، فونت و بافت اختصاصی.

۳. معماری کلی پروژه

کد پروژه به پنج لایه‌ی مجزا تقسیم شده که هرکدام مسئولیت مشخصی دارند. این جداسازی باعث می‌شود تغییر در یک لایه (مثلاً نحوه‌ی رسم صفحه) هیچ تأثیری بر منطق قوانین بازی نداشته باشد و برعکس.

لایه	پوشه	توضیحات
Model	/model	کلاس‌های داده‌ای ساده: Card، Fighter/Hero/Sidekick، Deck، Map، Tile. این لایه فقط وضعیت و دسترسی‌های ساده دارد و هیچ قانونی از بازی در آن پیاده نشده است.
Engine	/engine	قطعات واقعی قوانین بازی: Effect (کاری که یک کارت انجام می‌دهد)، Condition (شرط اعمال‌شدن یک اثر)، Query (مقداری محاسبه‌شده که یک اثر می‌خواند، مثلاً تعداد کمک‌کارها) و gameData (زمینه‌ای مشترک که بین این سه رد و بدل می‌شود).

لایه	پوشه	توضیحات
Factory	/factory	تبدیل فایل‌های JSON زیر data/ به اشیای واقعی Hero/Card/Map/Effect ... تنها لایه‌ای که فرمت داده روی دیسک را می‌شناسد؛ هر خطای احتمالی را می‌گیرد و به FactoryException تبدیل می‌کند.
Controller	/controller	GameManager (موتور قوانین / ماشین حالت نوبت‌ها و نبردها)، SceneManager (صحنه‌ی فعال)، PlayerSelectionManager (انتخاب قهرمان پیش از شروع بازی)، AudioManager/FontManager/TextureManager (کش منابع)، و app (حلقه‌ی اصلی raylib).
View	/view	صحنه‌های رسم‌شده با raylib (MainScene، HeroSelectionScene، GameScene) و اجزای رابط کاربری کوچک (Button، Particle). صحنه‌ها فقط وضعیت GameManager را می‌خوانند و رسم می‌کنند؛ هیچ قانونی در آن‌ها پیاده نشده است.

جریان داده برای قوانین بازی همواره یک‌طرفه است: فایل Factory ← JSON ← شیء Engine ← Model (Effect/Condition/Query) ← اعمال نتیجه توسط GameManager ← خواندن وضعیت و رسم آن توسط View. این جهت‌دهی مشخص، ردیابی خطا و افزودن قابلیت جدید را ساده می‌کند، چون هر تغییر مسیر مشخصی برای عبور از لایه‌ها دارد.

۴. الگوهای طراحی استفاده‌شده

۴.۱ Factory

به ازای هر نوع داده، یک کلاس Factory مجزا وجود دارد: HeroFactory، CardFactory، DeckFactory، MapFactory، TileFactory، SidekickFactory، EffectFactory، ConditionFactory، QueryFactory و HeroInfoFactory. همه‌ی این Factoryها ساختار یکسانی دارند: ورودی خام (یک JSON یا مسیر فایل) را می‌گیرند، یک شیء Model/Engine می‌سازند و بدنه را در try/catch قرار می‌دهند تا در صورت خطا، یک FactoryException با اطلاعات کامل پرتاب کنند. با این طراحی، افزودن یک قهرمان یا کارت جدید تنها به افزودن یک فایل JSON زیر data/ نیاز دارد، بدون هیچ تغییری در کد C++.

۴.۲ Observer

رابط IGameObserver به GameScene اجازه می‌دهد به رویدادهایی مانند onGameStarted، onCardPlayed، onFighterDamaged و onCombatResolved که توسط GameManager فراخوانی می‌شوند واکنش نشان دهد؛ بدون این‌که GameManager هیچ اطلاعی از لایه‌ی رسم داشته باشد. این جداسازی دقیقاً همان مرز بین View و Model/Engine را که در بخش معماری توضیح داده شد، در عمل پیاده می‌کند.

۴.۳ چندریختی به سبک Strategy

کلاس‌های Effect، Condition و Query هرکدام یک کلاس پایهی انتزاعی هستند که پیاده‌سازی‌های مشخص زیادی دارند (مثلاً drawEffect، moveEffect، dmgEffect برای Effect، یا isNearEnemyCondition و wonTheWarCondition برای Condition) و در زمان بارگذاری، بر اساس فیلد type یا name در JSON، توسط Factory مربوطه ساخته می‌شوند. این یعنی رفتار هر کارت، بدون نیاز به شاخه‌های شرطی گسترده در کد، صرفاً با ترکیب این اشیای چندریخت تعیین می‌شود.

۴.۴ Singleton (به سبک Meyer's)

PlayerSelectionManager تنها Singleton عمدی پروژه است که برای انتقال اطلاعات «کدام بازیکن کدام قهرمان را انتخاب کرده» از صفحه‌ی انتخاب قهرمان به داخل بازی استفاده می‌شود. این Singleton به صورت یک static محلی داخل یک متد کلاس پیاده شده، نه یک متغیر سراسری (global) عمومی؛ این انتخاب از مشکلات رایج Singleton‌های کلاسیک (ترتیب مقداردهی اولیه‌ی نامشخص) جلوگیری می‌کند.

۴.۵ تزریق وابستگی (Dependency Injection)

کلاس app مالک AudioManager، FontManager، TextureManager و SceneManager به صورت unique_ptr است و اشاره‌گر خام آن‌ها را به هرکس که نیاز داشته باشد پاس می‌دهد؛ بدون این‌که هیچ‌کدام از این‌ها متغیر سراسری باشند. این رویکرد، تست‌پذیری و کنترل چرخه‌ی عمر منابع (به‌ویژه بافت‌ها و صداها (raylib)) را بهبود می‌دهد.

۵. مدیریت استثناها (Exception Handling)

به جای این‌که خطاها بی‌صدا نادیده گرفته شوند یا برنامه با std::terminate() و بدون پیام مفید متوقف شود، یک سلسله‌مراتب استثنای اختصاصی در include/utility/exceptions.h تعریف شده است:

استثنا	توضیح
AppException (پایه)	ریشه‌ی سلسله‌مراتب؛ همه‌ی استثناهای پروژه از این کلاس مشتق می‌شوند.
FileException	امکان باز کردن یا لیست کردن فایل/پوشه وجود ندارد.
JsonParseException	فایل باز شد اما محتوای آن JSON معتبر نیست.
DataFormatException	JSON با موفقیت پارس شد اما فیلد لازم وجود ندارد یا نوع آن اشتباه است.
FactoryException	یک Factory در ساخت شیء ناموفق بود؛ یکی از سه استثنای بالا را با اطلاعات «چه چیزی در حال ساخته شدن بود» بسته‌بندی می‌کند.

هر لایه هنگام عبور استثنا به لایه‌ی بالاتر، اطلاعات بیشتری به آن اضافه می‌کند و لایه‌ی صحنه (Scene) جایی است که برنامه واقعاً بازیابی می‌شود، نه فقط لاگ می‌کند:

- utility/file.cpp (توابع load() و listFiles()) در پایین‌ترین سطح، استثنای FileException یا JsonParseException پرتاب می‌کند.
- هر Factory::create() خطاهای داخلی خودش را می‌گیرد و یک FactoryException با نام‌گذاری دقیق (مثلاً «ساخت قهرمان Dracula ناموفق بود: فیلد name در stats.json وجود ندارد») دوباره پرتاب می‌کند.
- GameManager::createFromSelection() اطلاعات «نقشه‌ی X در حال آماده‌سازی بود» را به هر خطای Factory اضافه می‌کند.
- SceneManager::changeScene() هر AppException ناشی از ساخت یک صحنه را می‌گیرد، آن را لاگ می‌کند و به جای کرش کل برنامه، به منوی اصلی برمی‌گردد؛ اگر ساخت خود منوی اصلی هم شکست بخورد، استثنا مجدداً پرتاب می‌شود.
- main() آخرین لایه‌ی محافظتی است که هر خطای رسیدگی نشده را می‌گیرد و با پیامی واضح خارج می‌شود، به جای یک crash بی‌نام‌ونشان.

۶. نحوه‌ی Build و اجرای پروژه

پیش‌نیازها: CMake نسخه‌ی ۳.۱۶ به بالا، یک کامپایلر با پشتیبانی از C++17 و (در لینوکس) هدرهای توسعه‌ی X11/OpenGL که raylib به آن‌ها نیاز دارد (libx11-dev، libxrandr-dev، libxinerama-dev، libxcursor-dev، libxi-dev، libgl-dev). کتابخانه‌های raylib و nlohmann/json به‌صورت خودکار توسط CMake از طریق FetchContent دریافت می‌شوند و نیازی به نصب دستی آن‌ها نیست.

```
mkdir build && cd build
cmake .. -DCMAKE_BUILD_TYPE=Release
cmake --build . -j$(nproc)
./CardGameEngine
```

در زمان build، پوشه‌های /assets و /data به‌طور خودکار کنار فایل اجرایی کپی می‌شوند (طبق تنظیمات CMakeLists.txt)، بنابراین برنامه بدون نیاز به تنظیم مسیر دستی، بلافاصله پس از build قابل اجراست.

۷. ساختار پوشه‌ها و توضیح فایل‌به‌فایل

در این بخش، محتوای هر پوشه‌ی اصلی پروژه و نقش فایل‌های مهم آن به تفصیل توضیح داده می‌شود.

۷.۱ include/controller و src/controller

فایل	توضیح
app.h	کلاس اصلی برنامه؛ مالک SceneManager و مدیرهای منابع (Audio/Font/Texture)، راه‌انداز پنجره‌ی raylib و اجراکننده‌ی حلقه‌ی اصلی برنامه (run). فقط هدر.
GameManager.h/.cpp	موتور قوانین بازی؛ ماشین‌حالت نوبت‌ها، محدودیت دو اکشن در هر نوبت، حرکت (Maneuver)، نبرد هم‌زمان دو کارته (startCombat/resolveCombat)، کارته‌های Scheme (playCard)، مکانیزم انتخاب معلق برای درخواست از بازیکن (انتخاب خانه یا کارت)، تشخیص برنده، و توانایی‌های ویژه‌ی شخصیت‌ها. createFromSelection () نقطه‌ی ورودی ساخت یک مسابقه‌ی آماده از روی انتخاب PlayerSelectionManager است.
SceneManager.h	مدیریت صحنه‌ی فعال و جابه‌جایی امن بین صحنه‌ها با پشتیبانی از بازیابی خطا (رجوع کنید به بخش مدیریت استثناها). فقط هدر.
PlayerSelectionManager.h	Singleton نگه‌دارنده‌ی انتخاب قهرمان هر بازیکن پیش از شروع بازی. فقط هدر.
AudioManager / FontManager / TextureManager	کش منابع صوتی، فونت و بافت که هرکدام فقط یک‌بار بارگذاری و در فراخوانی‌های بعدی از کش بازگردانده می‌شوند.

۷.۲ include/model و src/model

این پوشه کلاس‌های داده‌ای خالص بازی را در بر دارد: Card، Tile، Fighter/Hero/Sidekick، Deck، Map. به همراه enum‌های کمکی و ساختار Player برای مرحله‌ی انتخاب قهرمان.

فایل	توضیح
fighter.h/.cpp	کلاس پایه برای هر چیزی که می‌تواند روی نقشه بایستد و بجنگد: نام، تصویر، سلامتی/حداکثر سلامتی، حرکت، نوع حمله (نزدیک/دور) و خانه‌ی فعلی. Hero و Sidekick هر دو از آن مشتق می‌شوند.
hero.h/.cpp	قهرمان یک بازیکن: کمک‌کارهای اضافه و Deck اختصاصی او.
sidekick.h/.cpp	کمک‌کار یک قهرمان (مثلاً خواهر Dracula یا Watson). فقط getFighterType () را روی sidekick تنظیم می‌کند.
card.h/.cpp	یک کارت واحد: نام، تصویر، نوع (attack/def/multipurpose/event)، زمان‌بندی رویداد، محدودیت اجراکننده، مقادیر حمله/دفاع/Boost و فهرست Effect‌های آن.
deck.h/.cpp	دسته‌ی کارت‌های یک قهرمان (draw pile / discard pile / دست بازیکن) با متدهای draw ()، shuffle () و discard ().
map.h/.cpp	صفحه‌ی بازی: مجموعه‌ای از Tile‌ها با کلید id، به همراه پرس‌وجوهایی مانند «خانه‌های درون فاصله‌ی N از خانه‌ی X» و «چه کسی روی خانه‌ی Y ایستاده»

فایل	توضیح
	که در حرکت، نبرد و محاسبه‌ی برد اثرات استفاده می‌شوند.
tile.h/.cpp	یک خانه‌ی صفحه: id، همسایه‌ها، تگ‌ها (startPoint، secretPassage)، ناحیه‌ها و موقعیت آن روی صفحه‌نمایش.
InfoHero.h	داده‌ی نمایشی صرف برای صفحه‌ی انتخاب قهرمان (نام، نقش، آمار، توضیحات، متن توانایی، رنگ تم)؛ مجزا از کلاس اصلی Hero تا این اطلاعات نمایشی مدل درون‌بازی را حجیم نکند.
Player.h	انتخاب قهرمان یک بازیکن در مرحله‌ی انتخاب: اندیس بازیکن، شناسه‌ی HeroList انتخاب‌شده، نام نمایشی. توسط PlayerSelectionManager استفاده می‌شود.
heroList.h	enum class HeroList شامل Dracula و SherlockHolms؛ نام‌ها دقیقاً با نام پوشه‌های /data /مطابقت دارند تا Factory‌ها بتوانند مستقیماً از آن استفاده کنند.
typeOfAttack.h	enum نوع حمله: melee یا ranged.
typeOfCard.h	enum نوع کارت: def، multipurpose، attack یا event.
typeOfEvent.h	enum زمان فعال‌شدن اثر یک کارت: before_combat، during_combat، after_combat، start_turn_self، all_the_time و none.
typeOfFighter.h / typeOfPerformer.h	enum‌های کمکی برای فیلتر کردن جنگجویان (hero/sidekick/fighter) و مشخص‌کردن این‌که چه کسی مجاز به بازی‌کردن یک کارت است.

include/engine V.۳

قطعات سازنده‌ی قوانین بازی؛ همه سرآمد (header-only) هستند و کلاس‌های نهایی توسط EffectFactory، QueryFactory و ConditionFactory از روی JSON ساخته می‌شوند.

فایل/پوشه	توضیح
gameData.h	ساختار زمینه‌ی مشترکی که به هر Effect::execute()/Condition::check()/Query::evaluate () پاس داده می‌شود: self، target، enemy، نقشه و callback‌های انتخاب بازیکن.
observer/observer.h	رابط GameObserver؛ قلاب الگوی Observer که GameManager آن را فرامی‌خواند (، onGameStarted، onCardPlayed، onFighterDamaged، onCombatResolved و ...) تا GameScene بدون وابستگی GameManager به لایه‌ی نمایش، از رویدادها مطلع شود.
effects/*.h	پیاده‌سازی اثرات کارت‌ها: آسیب (dmgEffect)، دفاع (defEffect)، تغییردهنده‌ی حمله/دفاع (modifierEffect)، کشیدن کارت (drawEffect)، حرکت (moveEffect)، حذف کارت (removeCardEffect)، غیرفعال‌کردن اثر/توانایی (removeEffectEffect)، دیدن دست حریف (seeHandEffect)، جابه‌جایی موقعیت (positionExchangeEffect)، افزودن به یک مقدار (addEffect)، تغییر مقدار کارت (changeValueEffect)، انتخاب خانه توسط بازیکن (choosePlaceEffect) و احیای کمک‌کار (reviveEffect).

فایل/پوشه	توضیح
conditions/*.h	شرط‌هایی مانند نزدیکی به دشمن، نزدیکی به هدف، نزدیکی یک قهرمان (نه هر جنگجو)، مجاورت دو جنگجوی هم‌تیمی، باخت یا برد آخرین نبرد، برد کل بازی، و تشخیص کارت‌های اختصاصی Sherlock/Watson.
queries/*.h	مقادیر محاسبه‌شده مانند شمارش جنگجویان از یک نوع خاص، خواندن مقدار Boost یک کارت، و شمارش کارت‌های حذف‌شده در طول بازی.

۷.۴ src/factory و include/factory

همه‌ی Factory‌ها ساختار یکسانی دارند: ورودی خام (یک JSON یا مسیر فایل) را می‌گیرند، یک شیء Model/Engine می‌سازند و بدنه را در try/catch قرار می‌دهند تا در صورت خطا، یک FactoryException با اطلاعات کامل پرتاب کنند.

فایل	توضیح
heroFactory.h/.cpp	ساخت Hero از data/<نام>/stats.json به همراه کمک‌کارها (از طریق SidekickFactory) و دسته‌ی کارت‌ها (از طریق DeckFactory).
sidekickFactory.h/.cpp	ساخت یک Sidekick از فایل data/<نام>/sidekicks/<sidekick>.json.
deckFactory.h/.cpp	ساخت Deck با بارگذاری هر فایل JSON کارت در پوشه‌ی deck/ یک قهرمان (از طریق CardFactory)، گسترش هر کارت به تعداد quantity و بُردن (shuffle) نتیجه.
cardFactory.h/.cpp	ساخت یک Card از JSON آن: مقادیر عددی، enum‌های اجراکننده/نوع/رویداد (با کمک magic_enum)، و فهرست اثرات آن (از طریق EffectFactory).
effectFactory.h/.cpp	بزرگ‌ترین Factory پروژه: نگاشت رشته‌ی effect.type در JSON (مانند dmg، move، draw_card، revive و ...) به زیرکلاس Effect متناظر، سپس اتصال Condition/ها/Query‌های آن.
conditionFactory.h/.cpp	نگاشت نام یک شرط به زیرکلاس Condition متناظر؛ برای نام ناشناخته nullptr برمی‌گرداند (به معنای «بدون شرط»).
queryFactory.h/.cpp	نگاشت نام یک Query به زیرکلاس متناظر و اتصال شرط‌های آن.
mapFactory.h/.cpp	ساخت Map از data/maps/<نام>.json: اعتبارسنجی فیلدهای لازم (name، tiles) و ساخت هر Tile از طریق TileFactory.
tileFactory.h/.cpp	ساخت یک Tile از JSON آن: id (اجباری)، همسایه‌ها، تگ‌ها، ناحیه‌ها و موقعیت روی صفحه‌نمایش.
heroInfoFactory.h/.cpp	ساخت InfoHero برای صفحه‌ی انتخاب قهرمان از فایل view.json، با اعتبارسنجی کامل فیلدهای لازم.

۷.۵ src/view و include/view

فایل	توضیح
Scene.h	کلاس پایه‌ی انتزاعی هر صحنه: Draw()/Update()/onExit()/onEnter()، به همراه دسترسی مشترک به مدیرهای صوت/فونت/بافت و SceneManager.

فایل	توضیح
MainScene.h/.cpp	منوی اصلی: عنوان، نقل قول‌ها و دکمه‌های شروع بازی، مشاهده‌ی مجموعه، تنظیمات و خروج؛ همچنین محرک افکت ذرات پس‌زمینه.
HeroSelectionScene.h/.cpp	انتخاب قهرمان برای هر بازیکن (از طریق HeroInfoFactory/InfoHero) و پخش صداهای مرتبط به ترتیب، سپس واگذاری کنترل به GameScene.
GameScene.h/.cpp	صفحه‌ی اصلی بازی؛ ساخت یک GameManager واقعی از طریق createFromSelection، رسم نقشه/دست کارت‌ها/دکمه‌های اکشن، و گوش‌دادن به رویدادها به عنوان IGameObserver برای همگام نگه‌داشتن نمایش (لاگ رویداد، نوار سلامتی و ...) با وضعیت بازی.
ui/Button.h، ui/Particle.h	ساختارهای ساده‌ی رابط کاربری: دکمه‌ی قابل کلیک با مستطیل برخورد و متن، و ذره‌ی پس‌زمینه با موقعیت/سرعت/شعاع/آلفا.
enums/ScenesType.h، TextureID.h، FontID.h، MusicID.h، SoundID.h	enumهای کمکی نمایش: صحنه‌ی فعال (ScenesType)، شامل سه صحنه‌ی هنوز پیاده‌نشده‌ی (HowToPlay/Settings/Collection)، شناسه‌ی بافت‌ها، فونت‌ها، موسیقی و صداها.

۷.۶ include/utility و src/utility

فایل	توضیح
exceptions.h	سلسله‌مراتب استثنای اختصاصی پروژه (رجوع کنید به بخش ۵) به همراه توابع کمکی json_util برای اعتبارسنجی فیلدهای JSON.
file.h/.cpp	دو تابع پایه: load(path) برای باز و پارس کردن یک فایل JSON، و listFiles(folder) برای لیست کردن فایل‌های یک پوشه؛ پایین‌ترین سطحی که می‌تواند FileException یا JsonParseException پرتاب کند.

۷.۷ include/libraries

کتابخانه‌های تک‌فایلی شخص ثالث که مستقیماً در پروژه قرار داده شده‌اند (نه از طریق CMake دریافت می‌شوند): magic_enum.hpp برای تبدیل enum به رشته و برعکس در زمان کامپایل (بدون نیاز به نگاشت دستی)، و IconsFontAwesome6.h برای کدهای آیکون 6 Font Awesome در فونت رابط کاربری.

۷.۸ /data

داده‌های بازی به‌طور کامل JSON هستند و هیچ محتوایی در C++ هاردکد نشده است:

مسیر	توضیح
Dracula/stats.json و SherlockHolms/stats.json	سلامتی، حرکت، نوع حمله، فهرست کمک‌کارها.
Dracula/view.json و SherlockHolms/view.json	اطلاعات نمایشی صفحه‌ی انتخاب قهرمان (نام، توضیحات، متن توانایی، رنگ تم).
Dracula/sidekicks/*.json و SherlockHolms/sidekicks/*.json	آمار هر کمک‌کار؛ یک فایل به ازای هر کمک‌کار.
Dracula/deck/*.json و SherlockHolms/deck/*.json	یک فایل به ازای هر کارت یکتای دسته‌ی هر قهرمان (شامل مقدار quantity برای بُردن).

مسیر	توضیح
maps/baskervilleManor.json و maps/sarpedonMap.json	نام نقشه و فهرست کامل خانه‌ها؛ برای دو نقشه‌ی موجود در پروژه.

۸. جمع‌بندی و نتیجه‌گیری

در این پروژه، یک موتور بازی کارتی برای Unmatched با تمرکز بر جداسازی مسئولیت‌ها میان لایه‌های Model، Factory، Controller، Engine و View پیاده‌سازی شد. رویکرد داده‌محور بودن محتوای بازی (خواندن قهرمان‌ها، کارتها و نقشه‌ها از JSON) باعث شد افزودن محتوای جدید بدون تغییر در کد C++ ممکن باشد. همچنین سلسله‌مراتب استثنای اختصاصی، مدیریت خطا را در تمام لایه‌ها به شکلی یکپارچه و قابل‌ردیابی درآورد و لایه‌ی صحنه امکان بازیابی نرم از خطا را بدون کرش کل برنامه فراهم کرد.

استفاده از الگوهای طراحی استاندارد مانند Singleton، Strategy، Observer، Factory و Dependency Injection، پروژه را برای توسعه‌های آینده (مانند افزودن قهرمان‌های جدید، نقشه‌های بیشتر یا اثرات کارتی پیچیده‌تر) آماده نگه داشته است؛ چراکه هر یک از این الگوها مرز مشخصی میان مسئولیت‌ها ایجاد کرده و افزودن قابلیت جدید را به افزودن یک کلاس یا فایل JSON، به جای تغییر کدهای موجود، تبدیل می‌کند.

محدودیت‌های شناخته‌شده

- برخی صحنه‌ها مانند Settings، HowToPlay و Collection هنوز پیاده‌سازی نشده‌اند و فقط در enum مربوط به نوع صحنه (ScenesType) رزرو شده‌اند.
- پروژه در حال حاضر فقط دو قهرمان (Dracula و Sherlock Holmes) و دو نقشه دارد؛ افزودن قهرمان‌های بیشتر نیازمند فایل‌های JSON مشابه است، نه تغییر کد.
- تست‌های خودکار (unit test) در این export مشاهده نشد؛ صحت‌سنجی عمدتاً از طریق اعتبارسنجی JSON در زمان بارگذاری و سلسله‌مراتب استثنا انجام می‌شود.

۹. منابع (Resources)

در طراحی، پیاده‌سازی و مستندسازی این پروژه از منابع، کتابخانه‌ها و مراجع زیر استفاده شده است:

۹.۱ کتابخانه‌ها و ابزارهای نرم‌افزاری

منبع	توضیح / کاربرد در پروژه
raylib – https://www.raylib.com	کتابخانه‌ی سبک و ساده‌ی گرافیکی برای رندر ۲ بعدی، مدیریت پنجره، ورودی و صدا؛ زیربنای کل لایه‌ی View و مدیرهای منابع پروژه.
nlohmann/json – https://github.com/nlohmann/json	کتابخانه‌ی سرآمد C++ برای خواندن/نوشتن JSON؛ مبنای تمام Factoryها برای تبدیل فایل‌های data/ به اشیای C++.
CMake – https://cmake.org	ابزار مدیریت build؛ پی‌کربندی FetchContent برای دریافت خودکار raylib و nlohmann/json، و کپی خودکار پوشه‌های assets/ و data/.
magic_enum – https://github.com/Neargye/magic_enum	کتابخانه‌ی تک‌فایلی برای تبدیل enum به رشته و برعکس در زمان کامپایل، بدون نیاز به نگاشت دستی؛ در سراسر Factoryها برای خواندن مقادیر enum از JSON استفاده شده است.
IconsFontAwesome6.h – https://github.com/juliettef/IconFontAwesome6.h	مجموعه‌ی آیکون‌های Font Awesome 6 برای استفاده در فونت رابط کاربری بازی.

۹.۲ مرجع بازی و مفاهیم طراحی

منبع	توضیح
Unmatched Board Game – Restoration Games	بازی رومیزی اصلی که این پروژه پیاده‌سازی دیجیتال آن است؛ مرجع قوانین حرکت، نبرد، دسته‌ی کارت و شخصیت‌های Dracula و Sherlock Holmes.
Design Patterns: Elements of Reusable Object-Oriented Software – Gamma, Helm, Johnson, Vlissides (Gang of Four)	مرجع کلاسیک الگوهای طراحی شیء‌گرا (Factory Method، Observer، Singleton، Strategy) که مبنای معماری فصل ۴ این گزارش قرار گرفته‌اند.
cppreference.com – https://en.cppreference.com/w/cpp/17	مرجع استاندارد زبان و ویژگی‌های C++17 (مانند std::unique_ptr، std::optional و structured bindings) که در سراسر کد پروژه استفاده شده‌اند.

پایان گزارش.